

Unit-2

Data structure and Functions

* String-

String is collection or group of characters. A single character also considered as string in python because Python does not supports char data type.

In Python string (**str**) is a built-in data type.

String literals in python are surrounded by either single quotation marks, or double quotation marks or Triple double quotation.

Strings are immutable, hence elements of a String cannot be changed once it has been assigned.

Ex. 'Hello' or "Hello" or """Hello"""

```
s = "Hello"
print(s)
print("Datatype of s",type(s))
```

Output- Hello

Datatype of str <class 'str'>

Like many other popular programming languages, strings in Python are arrays of bytes representing Unicode characters.

Square brackets can be used to access elements of the string.

```
a = "Hello World"
print(a[1])
```

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		W	o	r	l	d

Output- e

In above program we accessed single character present in index number **1** as **a[1]** where **a** is variable in Which our string is stored.

Slicing of String –

we can return a specific range of characters present in string by using the **slicing** technique.

In slicing we have to Specify the start index and the end index, separated by a colon, to return a part of the string.

Syntax for slicing

variable [start index number : end index number]

where

variable: In which our string is stored

start index number: from which we want to start slicing

end index: end index-1 stop slicing

Example: Get the characters from position 1 to position 5 (not included):

```
b="Sangola"
print(b[1:5])
```

	0	1	2	3	4	5	6
b	S	a	n	g	o	l	a

Output: ango

In above program string stored in variable **b** we sliced our string as, **b[1:5]**

Where,

1 – is start index number from slicing starts

5- is end index number(not included) i.e **end-1 (5-1) = 4** where slicing

stopsIn simple words slicing starts from index 1 and stops at index 4

Negative Indexing : We can use negative indexes to start the slice from the end of the string.

```
b="Sangola"
print(b[-2:-5:-1])
```

	0	1	2	3	4	5	6
b	s	a	n	g	o	l	a
	-7	-6	-5	-4	-3	-2	-1

Output – log

In above program we performed negative indexing as, **b[-2: -5 : -1]**

Where,

-2 : is start of slicing

-5 : (-4) is end of slicing

-1: is step size

len () method :

To get the length of any string we use the len() method.

Syntax :- len(variable name)

```
x="Python"
print("Length of first string:",len(x))
y="Java"
d=len(y)
print("Length of second string-",d)
```

Output:-

Length of first string: 6

Length of second string: 4

Concatenation of Two or More Strings :

Joining of two or more strings into a single one is called concatenation.

The + operator does this in Python. Simply writing two string literals together also concatenates them.

```
s1 = 'Hello' s2 ='World!'
st=s1+" "+s2
print('Concatinated string is ', st)
```

Output:

Concatinated string is Hello World!

String Membership Test : We can test if a substring exists within a string or not, using the keyword **in** .in operator returns **True** value if substring **found**s in main string.

in operator returns **False** value if substring **not found**s in main string.

```
>>> 'n' in 'sangola'
```

Output : True

Ex. 1)

```
s1='Hello world'
s2='world'
print(s2 in s1)
```

O/P- True

2)

```
s1='Hello world'
s2='world'
print(s2 not in s1)
```

O/P- False

upper() and lower() methods :

upper() : This method converts all small letters into capital letters of a string.

lower () : This method converts all capital letters into small letters of a string.

```
s1 = 'SaNgolA'
print("To convert small letters into upppercase-",s1.upper())
s2="COLLEGE"
print("To convert capital letters into smallcase-",s2.lower())
```

Output:

To convert small letters into upppercase - SANGOLA

To convert capital letters into smallcase – college

split() method:

The **split()** method splits the string into substrings if it finds instances of the separator

```
a = "Hello World!"  
print(a.split(",")) # returns ['Hello', ' World!']
```

In above program we used , (comma) as a delimiter in split() method that's why our string is divided in two parts first part is string before comma and second part is string after comma.

* List :

A List is similar to an Array that contains group of elements or Items. Just like an array, a List can store elements.

But, there is one major difference between array and List is,

An array can store multiple elements of **Same** data type where a List can store multiple items of **different** data types.

Hence list are more versatile and useful than array. Perhaps List are most used data types in python.

-Elements or items of a list are ordered and changeable .

-List allows duplicate members.

-In Python lists are written with square brackets.

-One list can contain another list as Element which is called as **Nested List**.

Example.

```
x = [1,3.5,"apple",[11,22]]  
print(x)  
print("Datatype of x",type(x))
```

Output: [1,3.5,"apple",[11,22]]
Datatype of x <class 'list'>

In above program multiple elements of different data types are written inside square brackets so this collection is called List also note that we included a sub list or nested list [11,22] in our main list

We can access the list items by referring to the index number as

```
y = ["apple", "banana", "cherry"]  
print(y[2])
```

	0	1	2
y	"apple"	"banana"	"cherry"

Output : "cherry"

Slicing the List :

We can access specific range of items in a list by using the slicing operator :(colon).

```
student=[1,"Abc",60,66,59,44,70,75]
print(student)
print(student[0:5:1])#from 0th index to 5-1 index by 1 step
```

0	1	2	3	4	5	6	7
1	"Abc"	60	66	59	44	70	75

In above program we sliced the list as **student[0 : 5 : 1]** where **0** is start index number from slicing will start **5** is end index number which is not included i.e. (**5-1=4**) slicing stops at 4th index number and **1** is step size
So our output will be [1, 'Abc', 60, 66, 59]

If we take step size 2 then elements of 0th, 2nd, and 3rd index numbers will be return i.e. [1,60,59]

len() method :

the len() is used to count and return the length of List .

syntax- **len(list variable)**

```
x=[3,6,2,5,8,9, 'Max' ]
print(x)
print("Length of x",len(x))
```

output: [3, 6, 2, 5, 8, 9, 'Max']
Length of x 7

insert() method :

The insert() used to add an element in List at specific index number.

Syntax: **list.insert(indexnumber,element)**

```
x=[3,6,2,5,8,9, 'Max' ]
x.insert(2,"python")
print(x)
```

Output: [3, 6, 'python', 2, 5, 8, 9, 'Max']

In above program we inserted an element **"python"** at 2nd index number of our List x .

remove() method :

remove() method removes specific element from List.

Syntax – **list.remove(element)**

```
x=[3,6,2,5,8,9, 'Max' ]
x.remove(6)
print(x)
```

Output: [3, 2, 5, 8, 9, 'Max']
By remove(6) -> 6 element removed from list

pop() method :

pop() method pops out the last elements of List.

Syntax – **list.pop()**

```
x=[3,6,2,5,8,9, 'Max' ]
x.pop()
print(x)
```

Output: [3, 6, 2, 5, 8, 9]
By pop() -> last element of List "Max" is removed

Sort() : Sorts List in ascending manner.

Note: all elements of list are of same data type to apply this method.

syntax – list.sort()

```
x=[3,6,2,5,8,9]
x.sort()
print(x)
```

output: [2, 3, 5, 6, 8, 9]

reverse() : Reverse the sequence of elements in the List

Syntax – list.reverse()

```
x=[3,6,2,5,8,9, 'Max' ]
x.reverse()
print(x)
```

Output : ['Max', 9, 8, 5, 2, 6, 3]

count() : returns number of occurrence of specific elements in list.

Syntax : list.count(element)

```
x=[3,6,2,5,8,9, 'Max' ]
print(x.count(3))
```

Output : 1
x.count(3) returns how many times the element **3** occurred in list **x**

copy() : copies one List into another List.

Syntax- variable=list.copy()

```
x=[3,6,2,5,8,9, 'Max' ]
v=x.copy()
print("copied list",v)
```

Output : copied list [3, 6, 2, 5, 8, 9, 'Max']

By v=x.copy() ,List present in x is copied into v

Appending and Extending lists in Python :

We can add one item to a list using the **append()** method or add several items using **extend()** method.

```
odd = [1, 3, 5]
odd.append(7)
print(odd)
odd.extend([9, 11, 13])
print(odd)
```

Output : [1,3,5,7]

[1,3,5,7,9,11,13]

Concatenation of two List : We can concatenate two or more List for that + operator is used.

```
m=["solapur"]
n=["kolhapur",'pune']
p=m+n
print("concatinated List",p)
```

Output:

concatinated List ['solapur', 'kolhapur', 'pune']

max() and min() methods :

max() returns biggest element present in List and min() returns smallest element from list.

Syntax:

max(list)

min(list)

```
x=[3,6,2,5,8,9]
print("Biggest element",max(x))
print("smallest element",min(x))
```

output:

Biggest element 9

smallest element 2

*** Tuple :**

A tuple is a sequence which stores group of elements or items.

Tuple and Lists are similar but the main difference Tuples are immutable(unchangeable) where as Lists are mutable.

-Tuple allows duplicate members.

-In Python tuples are written with **round** brackets.

-A Tuple can contain another tuple called as **Nested Tuple**.

-We can't modify tuple's elements.

Example to create and display a Tuple

```
y = ('Sangola', 1, 55, 3.1)
print(y)
print("Datatype of y", type(y))
```

Output:

('Sangola', 1, 55, 3.1)

Datatype of y <class 'tuple'>

Note: If we do not use any brackets to enclose elements then by default it is considered as Tuple
e.g. **x=2,1,5,7,'hello'** the elements not enclosed in brackets so it will be considered as Tuple.

Nested Tuple: A tuple inside tuple is called as nested tuple.

```
z=(1,2,'Max',(55,66))
print(z)
```

Output:(1, 2, 'Max', (55, 66))

In our program (55,66) is a nested tuple or We can say it sub tuple which is inside main Tuple.

Accessing Tuple Elements :

We can access Tuple elements by three ways,

a) By Indexing b) By Negative Indexing c) By slicing

let's see these three ways.

a) Accessing Tuple Element by Indexing :

In this technique We can use the index operator [] to access an item in a tuple, where the index starts from 0.

```
z=(1,2,'Max',(55,66))
print(z[2])
```

	0	1	2	3
z	1	2	'Max'	(55,66)

Output: **Max**

In above program we accessed the element of 2nd index number of tuple z as z[2] where 'Max' string is present.

b) Accessing Tuple Element by Negative Indexing :

Python allows negative indexing for its sequences.

The index of -1 refers to the last item, -2 to the second last item and so on.

```
z=(1,2,'Max',(55,66))
print(z[-3])
```

	0	1	2	3
z	1	2	'Max'	(55,66)
	-4	-3	-2	-1

Output: **2**

In above program we accessed element **2** of our tuple by using it's negative index number as z[-3].

c) Accessing Tuple Element by slicing :

We can access a range of items in a tuple by using the slicing operator colon : called slicing.

```
z=(1,2,'Max',(55,66))
print(z[1:3])
```

	0	1	2	3
z	1	2	'Max'	(55,66)

Output: (2, 'Max')

In above program we done slicing of Tuples as z[1:3] where **1** is an index number from which slicing will start 3 is end number(which is not included) i.e. 3-1(end-1) = **2** where slicing stops.

Methods To process Tuples

len() method : len() returns number of elements present in tuple.

Syntax- **len(tpl)**

```
y=('sangola',1,55,3)
print(y)
print("\n length of y",len(y))
```

Output: ('sangola', 1, 55, 3)
length of y 4

min() method : min() returns smallest element present in tuple.

Syntax- **min(tpl)**

```
x=(7,1,55,3)
print("minimum in x:",min(x))
```

Output: minimum in x:1

Note: min() method works only when all elements present in tuple are of same data type.

max() method : max() returns largest element present in tuple.

Syntax- **max(tpl)**

```
x=(7,1,55,3)
print("maximum in x:",max(x))
```

Output: maximum in x:55

Note: max() method works only when all elements present in tuple are of same data type.

count() method : It returns how many times the element 'p' is found in tuple.

Syntax- **tpl.count(p)**

```
z=(7,1,5,3,7)
print("Number of occurrence of 7:",z.count(7),"times")
```

Output : Number of occurrence of 7: 2 times

index() method : It returns the first occurrence of element 'p' in tuple. Also this method gives ValueError if 'p' is not found in tuple.

Syntax: **tpl.index(p)**

```
z=(7,1,5,3,7)
print("First time occurrence of 7 at:",z.index(7),"index Number")
```

Output: First time occurrence of 7 at: 0 Index Number

sorted() method : This method sorts the Tuple elements in ascending order.

Syntax- **sorted(tpl)**

Also we can sort the tuple elements in reverse order by using following syntax

Syntax- **sorted(tpl, reverse=True)**

```
z=(7,1,5,3,7)
print("Elements in Ascending order:",sorted(z))
print("Elements in Reverse order:",sorted(z, reverse=True))
```

Output: Elements in Ascending order: [1, 3, 5, 7, 7]

Elements in Reverse order: [7, 7, 5, 3, 1]

Converting list to a Tuple

```
x = [0, 1, 2]
print(tuple(x))
print("Data type of x",type(x))
```

Output: (0,1,2)

Data type of x <class 'list'>

Concatenation of two tuples :

```
m =(0, 1, 2)
n=(7,1,5,3,7)
p= m+n
print("concatenated tuple",p)
```

Output: (0, 1, 2, 7,1,5,3,7)

inserting element in tuple :

tuples are immutable. This means that elements of a tuple cannot be changed once they have been assigned. But, if the element is itself a mutable data type like list, its nested items can be changed.

We can also assign a tuple to different values (reassignment).

```
z=(5,2,7,9)
z=z+(10,)
print(z)
```

Output: (5,2,7,9,10)

inserting element in between tuple :

```
n=3
z=(5,2,7,9)
print(z)
z=z[:n]+(1,)+z[n:]
print("Tuple after inserting 1 in 3rd index-",z)
```

Output : (5, 2, 7, 9)

Tuple after inserting 1 in 3rd index- (5, 2, 7, 1, 9)

Deleting an element from tuple :

Program to delete an element from 2nd index number of tuple

```
n=2
z=(5,2,7,9)
z=z[:n]+z[n+1:]
print("Tuple z",z)
```

Oput : Tuple z (5, 2, 9)

Deleting Entire Tuple:

```
z=(5,2,7,9)
del z
```

Tuple Membership Test

We can test if an item exists in a tuple or not, using the keyword **in**.

```
my_tuple = ('a', 'p', 'p', 'l', 'e',)

# in operation
print('a' in my_tuple)
print('b' in my_tuple)

# not in operation
print('g' not in my_tuple)
```

Output: **True**
False
True

* Dictionaries :

A dictionary represents a group of elements in the form of Key-Value pair. In the Dictionary first element Considered as **Key** and immediate next element taken as **Value**. The key and its value are separated by a **colon (:)**.

-A dictionary is a collection of elements which are **unordered, changeable**.

-**Duplicate Keys** are **not allowed** in Dictionaries , But **duplicate values allowed**.

-Python dictionaries are written with **curly brackets**.

1)Example to **create** and display a Dictionary :-

```
d={'Name':'Manish', 'salary':49000}
print("pairs in dictionary d are:",d)
print("data type of d",type(d))
```

Name:'Manish'	'salary':49000
↑	↑
Key	Value

Output: pairs in dictionary d are: {Name: 'Manish', 'salary': 49000}
data type of d <class 'dict'>

Dictionary Methods :

1)Create :-

```
d = {'apple': 3, 'banana': 5, 'orange': 2}
print(d)
```

In this example, d is a dictionary with three key-value pairs. The keys are 'apple', 'banana', and 'orange', and the values are 3, 5, and 2, respectively.

2)Update :- The dictionary is a mutable data type, and its values can be updated by using the specific keys. If the key is not available then a new entry will be added to the dictionary with the specified key-value pair. If the key is already available then old value will be replaced with new value.

```
# add a new item
d = {'apple': 3, 'banana': 5, 'orange': 2}
print(d)
d['grape'] = 4
print(d)
# modify an existing item
d = {'apple': 3, 'banana': 5, 'orange': 2}
print(d)
d['banana'] = 6
print(d)
```

3)Delete :-

```
# remove an item
d = {'apple': 3, 'banana': 5, 'orange': 2}
print(d)
del d['orange']
print(d)
```

Implementation of dictionary methods

1) dict.clear():-

The method **clear()** removes all items from the dictionary.

Syntax

dict.clear() Return Value

This method does not return any value.

Example

```
dict = {'Name': 'Zara', 'Age': 7};
dict.clear()
```

2) dict.copy():-

The method **copy()** returns a copy of the dictionary.

Syntax

dict.copy()

Example

```
dict1 = {'Name': 'Zara', 'Age': 7};  
dict2 = dict1.copy()  
print(dict2)
```

Output:-

```
{'Name': 'Zara' 'Age': 7, }
```

3) dict.fromkeys():-

The method **fromkeys()** creates a new dictionary with keys from seq and values set to value.

Syntax

dict.fromkeys(seq[, value]) Parameters

- **seq** – This is the list of values which would be used for dictionary keys preparation.
- **value** – This is optional, if provided then value would be set to this value Return Value

This method returns the list.

Example

```
seq = ('name', 'age', 'gender')  
dict = dict.fromkeys(seq)  
print (dict)  
dict = dict.fromkeys(seq, 10)  
print (dict)
```

Output:-

```
{'age': None, 'name': None, 'gender': None}  
{'age': 10, 'name': 10, 'gender': 10}
```

4) dict.get(key, default=None):-

The method **get()** returns a value for the given key. If key is not available then returns default value None.

Syntax

dict.get(key, default = None) Parameters

- **key** – This is the Key to be searched in the dictionary.
- **default** – This is the Value to be returned in case key does not exist.

Return Value

This method return a value for the given key. If key is not available, then returns default value None.

Example

```
dict = {'Name': 'Zabra', 'Age': 7}  
print (dict.get('Age'))  
print (dict.get('Education', "Never") )
```

Output:-

```
7
```

```
Never
```

5) **dict.items()**:-

The method items() returns a list of dict's (key, value) tuple pairs

Syntax dict.items() Return Value

This method returns a list of tuple pairs.

Example

```
dict = {'Name': 'Zara', 'Age': 7}
print dict.items()
```

Output:-

```
[('Age', 7), ('Name', 'Zara')]
```

6) **dict.keys()**:-

The method keys() returns a list of all the available keys in the dictionary.

Syntax

dict.keys()

Return Value

This method returns a list of all the available keys in the dictionary.

Example

```
dict = {'Name': 'Zara', 'Age': 7} print dict.keys()
```

Output:- ['Age', 'Name']

7) **dict.update(dict2)**:-

The method **update()** adds dictionary dict2's key-values pairs in to dict. This function does not return anything.

Syntax

dict.update(dict2) Parameters

- **dict2** – This is the dictionary to be added into dict.

Return Value

This method does not return any value.

Example

```
dict = {'Name': 'Zara', 'Age': 7} dict2 = {'Sex': 'female' } dict.update(dict2)
```

print (dict) **Output:-**

```
{'Age': 7, 'Name': 'Zara', 'Sex': 'female'}
```

8) **dict.values()**:-

The method values() returns a list of all the values available in a given dictionary.

Syntax

dict.values() Return Value

This method returns a list of all the values available in a given dictionary.

Example

```
dict = {'Name': 'Zara', 'Age': 7} print (dict.values())
```

Output:- [7, 'Zara']

9) **dict.pop(key)**:-

Removes the element with the specified key

Syntax

```
dict.pop(key)
```

Example

```
dict = {'Name': 'Zara', 'Age': 7} x=dict.pop('Age')  
print (x)
```

1) dict.popitem(key):-

Removes the last inserted key-value pair **Syntax**

```
dict.popitem()
```

Example

```
dict = {'Name': 'Zara', 'Age': 7} x=dict.popitem()  
print (x)
```

2) dict.setdefault(key,default-val):-

Returns the value of the specified key. If the key does not exist: insert the key, with the specified value

Key- key to be searched in dictionary

Default-val-key with a value i.e. default value inserted in dictionary if the key is not in the directory. If the default value is not provided it takes none value

Syntax

```
dict.setdefault(key,default-val)
```

Example

```
dict = {'Name': 'Zara', 'Age': 7} x=dict.setdefault('Age') print(x) x=dict.setdefault('Address')  
print (x)  
x=dict.setdefault('Address','Sangola') print (x)
```

Difference between Function and Method:

METHODS

- 1)A method implemented **inside** of a class. It belongs to an object.
- 2)A method is **called by only its associated object** (dependent).
i.e. object.method()
- 3)We **need to declare** the class.
- 4)Methods can **access all the data** provided in the given class.
- 5)A method always requires **'self'** as the first argument.

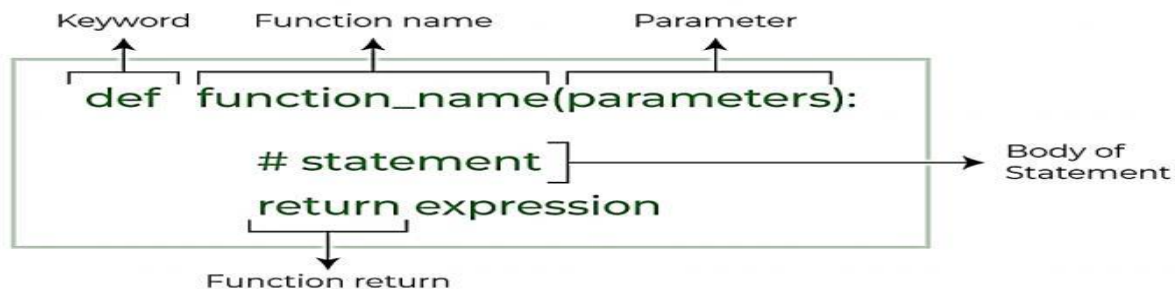
FUNCTIONS

- 1)A function implemented **outside** of a class. It does not belong to an object.
- 2)A function is an independent block of code that **is not associated with any objects**.
i.e. function (object)
- 3)We **don't need to declare** the class.
- 4)Functions can **only work** with the provided data.
- 5)A function doesn't take **'self'** as an argument.

Functions :-

If a **group of statements** is **repeatedly required** then it is not possible **to write these statements every time separately**. We have to **define these statements as a single unit** and we can call that **unit any number of times** based on our requirement **without rewriting**. This unit is **called function**.

A function contains a group of statements or a block of reusable code to perform a certain task as per requirement. Once a function is written, it can be used repeatedly. Python function defined with the def keyword.



def keyword - Every function in python should start with the keyword 'def'.

Ex: `def get_info():`

- **Name of the function**- Every function should be given a name, which can be used to call it.
- **Parenthesis**-After the name '()' parentheses are required
- **Parameters**- The parameters included within the parenthesis.
- **Colon symbol ':'**- The function definition always ends in a colon (:).
- **Body** -The body of the function should have an indentation of one level with respect to the line containing the 'def' keyword. The function body contains one or more statements that perform some actions. It can also use Pass keyword. Python uses the doctoring to generate documentation for the function automatically.

Ex:

```
def sample(): #function definition
print ("Hello world") #function body
sample() #function calling
```

The return statement:

The return statement is used to exit a function and go back to the place from where it was called also it returns some value. In python we can return multiple values from function.

Syntax of return:-

return [expression_list]

Ex.

```
def sub(p,q):
```



```

r=p-q
return r
m=sub(15,5)
print("\n Subtraction is",m)
print("\n Subtraction is=",sub(25,10))

```

Advantage of functions:

1. **Increasing modularity of code** – User to divide the large programs into small groups and solve them individually, so that read and debug program faster and better.
2. **Minimizes Redundancy** – Reducing duplication of code i.e. writing the same code multiple times
3. **Maximizes Code Reusability** – Once a function is created, it can be call many times.
4. **Information hiding** :- By using hide information, We can create more secure and maintainable code.
5. Make programs **simpler to read and understand**
6. It encourages us to **call the same method** with different **inputs multiple times**.

Types of Function :-

- Functions are of two types

1) Built-In functions

2) User Defined Functions:

1) Built-In functions : The Functions which **are already defined in python library** are called as Built-In functions. When we want to use, then we only have to call them.

Eg. print(), len() ,min(),max() ,type() etc. are built-in functions.

2) User Defined Functions:

The functions which **are defined by user** are called as user defined function.

To Use a user definedfunction we have to follow just Two steps.

a) Defining a Function: The function is **defined by user** is called as user defined function.To define a user defined function following syntax will be used.

```

def function _name(para1,para2,...):
    _____- logic- _____- ____
    - _____

```

Where,

def : is a keyword

Note: Logic of function should be write after indentation.

b) Calling a function : A function cannot run on its own, It **runs only when we call**

it. So our next step is to call the function. And the syntax is,

Function name(para1,para2,...)

A Simple program of user defined function.

```
def sample(): # 1. defining a function
    print("This is user defined function")

sample() # 2. Calling a function
```

Output:

This is user defined function

Function Parameter / Argument:

Parameters are **data values**, which are used in python program. Parameters can be literals, variables, arrays, lists, etc.

There are two Types of parameters

1) Actual Parameter: The parameters which **belongs to function call** are called as Actual Parameter.

2) Formal Parameter: The parameters which **belong to function definition** are called as Formal parameter.

Example of Actual and Formal Parameters :-

```
def ar(x,y): #x and y are formal parameters.
    print("addition is",(x+y))

p=20
q=30
ar(p,q) #call (p and q are actual parameters)
```

Output: addition is 50

Parameter Passing Techniques:

❖ **Pass by value:** In this technique a **copy of value present in Actual parameter** is send to the formal parameter.

If we **do changes in formal parameter** the changes can't reflect on **Actual parameter**.

```
def Byvalue(y):
    print("value of y",y)
    y=40
    print("new value of y",y)

x=25
Byvalue(x) #call
print("value of x",x)
```

Output: value of y 25
new value of y 40
value of x 25

Categories of function: Based on Arguments/Parameters and return statement function have 4 categories.

- 1) Function without Argument, without return value
- 2) Function without Argument, with return value
- 3) Function with Argument, without return value
- 4) Function with Argument, with return value

1) Function without Argument, without return value :

In this category, A function **does not** contains any Arguments/ Parameters and it **do not** returns any value from its definition.

```
def add():  
    a=10  
    b=5  
    c=a+b  
    print("addition=",c)  
  
add() #call
```

In this program add() does not accepts Any arguments and add() do not return Any value from its definition.

Output: addition= 15

2) Function without Argument, with return value :

In this category,

A function **does not** contains Arguments/ Parameters but it returns values from its definition.

```
def add():  
    a=2  
    b=5  
    c=a+b  
    return c  
x=add() #call  
print("addition=",x)
```

In this program add() **does not accepts Any arguments but add() returns**

A value present in variable c from its definition which is assigned to variable x present to right of function call.

Output: addition= 7

3) Function with Argument, without return value :

In this category, A function contains Arguments/ Parameters and it **do not** returns any value from its definition.

```
def add(p,q):  
    r=p+q  
    print("addition=",r)  
add(12,5) #call
```

In this program our function contains **Arguments p and q called as formal Parameters** which accepts values from **Actual parameters 12 and 3.**

Our function does not returns any value.

Output: addition= 17

4) Function with Argument, with return value :

In this category,

A function contains Arguments/ Parameters also it returns values from its definition.

```
def add(p,q):r=p+q
    return r

x=add(10,10) #call
print("addition=",x)
```

In this program add() contains Arguments p and q called as formal Parameters which accepts values from Actual parameters 10 and 10.

Also add() returns a value present in variable r from its definition which is assigned to variable x present to right of function call.

Output: addition=20

Types of arguments in Python:

1. Positional Arguments

- These are the most common type.
- Values are assigned to parameters **based on their position**.
- Ex.

```
def greet(name, age):
    print(f'Hello {name}, you are {age} years old.')
greet("Alice", 25) # 'name' = "Alice", 'age' = 25
```

2. Keyword Arguments

- You specify the parameter names explicitly when calling the function.
- Order doesn't matter.
- Ex.

```
def greet(age=30, name="Bob"):
    print(f'Hello {name}, you are {age} years old.')
greet()
```

3. Default Arguments

- You can give default values to parameters.
- If no value is provided for that parameter, the default is used.
- Ex.

```
def greet(name, age=18):
    print(f'Hello {name}, you are {age} years old.')

greet("Vishal")    # Uses default age = 18
greet("Avinash", 40) # Overrides default
```

4. Variable-length Positional Arguments (*args)

- Use *args to accept **any number of positional arguments**.

- args is a tuple.
- Ex.

```
def add(*numbers):
    return sum(numbers)
print(add(1, 2, 3))    # 6
print(add(10, 20))    # 30
```

5. Variable-length Keyword Arguments (**kwargs)

- Use **kwargs to accept **any number of keyword arguments**.
- kwargs is a dictionary.
- Ex.

```
def show_info(**info):
    for key, value in info.items():
        print(f'{key}: {value}')
show_info(name="Eve", age=28, country="USA")
```

Global, Local and Nonlocal Variables:

Global Variables : In Python, a **variable declared outside of the function** or in **global scope** is known as a global variable. This means that a global variable can be **accessed inside or outside of the function**.

```
x = 100      #global variable
def show():
    print("x inside:", x)

show() #call to show function
print("x outside:", x)
```

Output:

x inside: 100
x outside: 100

Local Variables :

A variable which is **declared inside the function's** body or in the **local scope** is known as a local variable.

```
def show():
    x = 10 #local variable
    print("x inside:", x)

show() #call to show function
#print("x outside:", x)
```

Output:

x inside: 10

Nonlocal Variables :-

Nonlocal variables are used in nested functions to modify variables in the outer functions scope. They are neither global nor local but belong to scope in between. Nonlocal variables can be accessed and modified by the inner nested function.

Ex.

```
def outer():
    z=30
    def inner():
        nonlocal z
        z=40
        print("Inner function=",z)
    inner()
    print("Outer function=",z)
outer()
```

O/P:-

Inner function= 40
Outer function= 40

Global keyword :-

The global keyword is used to create global variables from a no-global scope, e.g. inside a function.

Ex.

```
def myfunction(): #create a function:
    global x
    x = "hello"
myfunction() #execute the function:
print(x) #x should now be global, and accessible in the global scope.
```

Recursive() :-

It is a function that calls itself during its execution. which means a defined function can call itself.

Ex.Factorial Calculation :-This code defines a recursive function to calculate factorial of a number, where function repeatedly calls itself with smaller values until it reaches the base case.

```
def factorial(n):
    if n == 0: # Base case
        return 1
    else: # Recursive case
        return n * factorial(n - 1)
print(factorial(5))
```

Output :-

120

Explanation:

- **Base Case:** When **n == 0**, recursion stops and returns **1**.
- **Recursive Case:** Multiplies **n** with the factorial of **n-1** until it reaches the base case.

Advantages:-

- 1) We can reduce length of code and improves readability
- 2) We can solve complex problems very easily.

Anonymous/Lambda Function :

In Python, an anonymous function is a function that is defined **without a name**. While normal functions are defined using the `def` keyword in Python, anonymous functions are defined using the **lambda keyword**. Hence, anonymous functions are also called lambda functions.

Syntax :-

lambda arguments: expression

```
z= lambda x: x * 2
print(z(5))
```

Output: 10

In the above program, `lambda x: x * 2` is the lambda function.

Here `x` is the argument and `x * 2` is the expression that gets evaluated and returned.

lambdas with the filter() function in Python is a common pattern for filtering elements from a list (or any iterable) based on a condition.

- `filter(function, iterable)` returns an iterator with elements from iterable for which function returns True.
- A **lambda** is an anonymous inline function, often used with `filter` for concise filtering logic.

Example 1: Filter even numbers from a list

```
numbers = [1, 2, 3, 4, 5, 6]
# Using lambda with filter to get even numbers
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
```

```
print(even_numbers) # Output: [2, 4, 6]
```

Example 2: Filter words longer than 3 characters

```
words = ["cat", "dog", "elephant", "bee"]

long_words = list(filter(lambda w: len(w) > 3, words))

print(long_words) # Output: ['elephant']
```

How it works:

- `lambda x: x % 2 == 0` is a function that returns True if `x` is even.
- `filter()` applies this function to each item in `numbers` and keeps only those where the function returns True.

map() :-

- map(function, iterable) applies a function to **each item** in the iterable.
- It returns an iterator with the results.
- Lambdas are commonly used with map() for concise function definitions.

✓ **Example 1: Square each number in a list**

```
numbers = [1, 2, 3, 4, 5]
squared = list(map(lambda x: x ** 2, numbers))
print(squared) # Output: [1, 4, 9, 16, 25]
```

✓ **Example 2: Convert strings to uppercase**

```
words = ['apple', 'banana', 'cherry']

uppercased = list(map(lambda word: word.upper(), words))
print(uppercased) # Output: ['APPLE', 'BANANA', 'CHERRY']
```

Reduce() :-

- reduce() is used to apply a **rolling computation** to sequential elements of a list (or iterable), reducing them to a single value.
- It's part of the functools module in Python:
from functools import reduce
- Syntax:
reduce(function, iterable)

The function takes **two arguments** and is applied cumulatively to the items of the iterable.

✓ **Example 1: Sum all numbers in a list**

```
from functools import reduce
numbers = [1, 2, 3, 4, 5]
total = reduce(lambda x, y: x + y, numbers)
print(total)
# Output: 15
This is equivalent to: (((1 + 2) + 3) + 4) + 5
```

✓ **Example 2: Multiply all numbers**

```
from functools import reduce

numbers = [1, 2, 3, 4]

product = reduce(lambda x, y: x * y, numbers)

print(product) # Output: 24
```


Array:

“Array is also single variable but it can store multiple values at a time of same data type referred common name”

OR

“An Array is collection of elements or items having **same data type** referred by common name”

Ex.1)

```
x = ["Ford", "Volvo", "BMW"]  
print(x)
```

2) x = [5,10,3,4]

```
print(x)
```

3) x = [5,10,3,4]

```
print(x[2])
```

```
4) st=input("enter any value=")  
for n in st.split(','):   
    print([n])
```

Advantages of array :-

1. Random Access

- Arrays allow **direct access to elements** using their index, which makes retrieval operations very fast ($O(1)$ time complexity).

2. Cache Friendliness

- Arrays store elements in **contiguous memory locations**, which improves cache performance and speeds up data access compared to other data structures like linked lists.

3. Efficient for Numerical Computations (NumPy Arrays)

- **NumPy arrays** are optimized for numerical operations:
 - They store data as **primitive types** (e.g., integers, floats) instead of Python objects, reducing memory usage.
 - They support **vectorized operations**, allowing you to perform element-wise computations without explicit loops, which is much faster.

4. Fixed Size (Memory Efficiency)

- Arrays (e.g., from the array module or NumPy) have a **fixed size**, which can be more memory-efficient compared to Python lists that dynamically resize and allocate extra memory.

5. Multidimensional Support

- Arrays (especially NumPy arrays) support **multidimensional data structures**, making them ideal for working with matrices, tensors, and other higher-dimensional data.
-

6. Easy to Use

- Arrays are simple to declare and use, especially with Python's built-in array module or the NumPy library. They provide a clean and intuitive way to store and manipulate data.

7. Faster Iteration

- Iterating over arrays (especially NumPy arrays) is faster compared to Python lists because of their optimized implementation in C.

8. Support for Mathematical Operations

- NumPy arrays allow you to perform mathematical operations directly on the array (e.g., addition, subtraction, multiplication) without needing explicit loops.

9. Compatibility with Libraries

- Arrays (especially NumPy arrays) are widely used in Python libraries like **Pandas**, **Scikit-learn**, and **TensorFlow**, making them essential for data science, machine learning, and scientific computing.

Example: NumPy Array Advantages

```
import numpy as np
```

```
# Create a NumPy array
arr = np.array([1, 2, 3, 4, 5])
```

```
# Vectorized operation: Add 10 to each element
result = arr + 10
print(result) # Output: [11 12 13 14 15]
```

```
# Memory-efficient storage
print(arr.nbytes) # Shows memory usage in bytes
```

Summary

While Python lists are more flexible, arrays (especially NumPy arrays) are **faster, more memory-efficient, and better suited for numerical and scientific computations**.

Types of Array:

1) One Dimensional array:

“ The array having **only one subscript** is called as One dimensional array or vector”

Ex. `x = [3,7,1,4]`

```
print(x)
```

2) Two Dimensional array:

“ The array having **two subscripts** is called as Two dimensional array or matrix”

Ex. `x = [[3,7],[1,4]]`

```
print(x)
```

3) Multi-Dimensional array:

“The array having **more than two subscripts** is called as multi-dimensional array”

Ex. `x = [[3,7],[1,4],[1,2]]`
`print(x)`

importing array module :-

To use arrays in Python, you need to import the array module.

Example of importing the array module

```
import array
print(dir(array))
```

Output

```
['ArrayType', '__doc__', '__file__', '__loader__', '__name__', '__package__', '__spec__',
 '_array_reconstructor', 'array', 'typecodes']
```

In the above code example, we imported the array module and returned the list of all useful objects in the module.

Once imported, you can create an array using the `array.array()` function. Here's an example:

Ex.

```
import array
# Creating an array of integers
arr = array.array('i', [1, 2, 3, 4, 5])
# Printing the array
print("Array:", arr)
```

Explanation:

- 'i' specifies the type of elements in the array (e.g., 'i' for integers, 'f' for floats).
- [1, 2, 3, 4, 5] is the list of values to initialize the array.

This is a simple and efficient way to work with arrays in Python!

Creating an Array in Python

The module array has the following class:

Syntax:-

```
class array.array(typecode[, initializer])
```

Let us create an array using the above class.

Example of creating an array in Python

```
import array
numbers = array.array('i', [1,3,5,7])
print(numbers)
```

Output

```
array('i', [1, 3, 5, 7])
```

There are several attributes and methods that we can use on the array object that we created.

indexing and slicing on arrays :-

In Python, **indexing** and **slicing** are used to access elements or subsets of elements from **arrays** or **lists** (and other sequence types like strings and tuples). If you're working with arrays in Python, you might be using either **built-in lists** or **NumPy arrays**,

1. Indexing in Python :-

Indexing means accessing a single element by its position.

Python List Indexing:

```
arr = [10, 20, 30, 40, 50]
```

```
print(arr[0]) # First element → 10
print(arr[-1]) # Last element → 50
print(arr[2]) # Third element → 30
```

- **Positive index** starts from 0
- **Negative index** starts from -1 (last element)

NumPy Array Indexing:

```
import numpy as np
```

```
arr = np.array([10, 20, 30, 40, 50])
print(arr[1]) # 20
```

It works similarly to list indexing, but NumPy supports **multi-dimensional indexing**:

```
matrix = np.array([[1, 2], [3, 4]])
print(matrix[0, 1]) # Access row 0, column 1 → 2
```

2. Slicing in Python

Slicing retrieves a subset of elements using a colon syntax:

```
[start : stop : step]
```

Python List Slicing:

```
arr = [10, 20, 30, 40, 50]
```

```
print(arr[1:4]) # [20, 30, 40] (stop index is exclusive)
print(arr[:3]) # [10, 20, 30]
print(arr[::2]) # [10, 30, 50]
print(arr[::-1]) # [50, 40, 30, 20, 10] (reversed)
```

NumPy Array Slicing:

```
import numpy as np
```

```
arr = np.array([10, 20, 30, 40, 50])
print(arr[1:4]) # [20 30 40]
matrix = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])
```

```
# Slice rows 0 to 1, columns 1 to 2
print(matrix[0:2, 1:3])
# Output:
# [[2 3]
# [5 6]]
```

Useful Methods of array.array

Here are the most commonly used methods:

Method	Description
append(x)	Adds an item x to the end of the array
extend(iterable)	Appends items from an iterable
insert(i, x)	Inserts x at position i
remove(x)	Removes the first occurrence of x
pop([i])	Removes and returns item at index i (last item if not specified)
index(x)	Returns the first index of x
reverse()	Reverses the array in place
buffer_info()	Returns a tuple: (memory address, length of array)
count(x)	Returns the number of occurrences of x
tolist()	Converts the array to a regular Python list
fromlist(lst)	Appends items from the list lst
tobytes()	Converts the array to a bytes object
frombytes(bytes)	Reads items from a bytes object into the array

Example

import array

```
arr = array.array('i', [1, 2, 3])
arr.append(4)      # [1, 2, 3, 4]
arr.extend([5, 6]) # [1, 2, 3, 4, 5, 6]
arr.insert(0, 0)   # [0, 1, 2, 3, 4, 5, 6]
arr.remove(3)      # [0, 1, 2, 4, 5, 6]
print(arr.pop())   # 6, array becomes [0, 1, 2, 4, 5]
print(arr.index(4)) # 3
arr.reverse()      # [5, 4, 2, 1, 0]
print(arr.tolist()) # [5, 4, 2, 1, 0]
```